

5G Slice Validator

Professional Project Documentation

A reproducible tool for validating 5G slicing scenarios, analyzing logs, identifying root causes, and generating reports.

Project Type	5G SA Troubleshooting / Network Slice Validation
Main Technologies	Python, YAML, Regex Log Parser, Streamlit, Pytest
Outputs	CLI Results, JSON Report, Markdown Report, Dashboard View
Validation Result	All automated tests passed: 9/9

Executive Summary

The project checks whether UE, gNB, AMF, subscriber, and core-service settings are consistent. It then parses 5G-style logs to detect registration and PDU session failures, correlates evidence, and explains the most likely root cause with practical recommendations.

Document Sections

No.	Section
1	Project Overview
2	Problem Statement and Objectives
3	System Architecture
4	How the Tool Works
5	Main Modules
6	Supported Scenarios and Results
7	How to Run
8	Testing
9	Limitations and Future Work
10	Conclusion

1. Project Overview

5G Slice Validator is a Python-based troubleshooting tool designed to validate common 5G Standalone slicing scenarios. It reads configuration files and network logs, checks whether the main network components are aligned, and generates an explainable diagnosis when a failure occurs.

The project focuses on the parts that usually cause problems in a 5G lab setup: DNN, TAC, S-NSSAI/slice configuration, PDU session establishment, and the availability of core services such as SMF.

Main Idea

Instead of manually checking multiple YAML files and reading raw logs, the tool connects all evidence into one clear validation pipeline: configs + logs -> checks -> root cause -> report.

2. Problem Statement and Objectives

In 5G environments, a UE may fail to register or fail to establish a PDU session for several reasons. The failure might be caused by a wrong DNN, a TAC mismatch, an unsupported slice, or a core service that is down. Debugging these issues manually is time-consuming because the engineer must compare configuration files and interpret log cause codes.

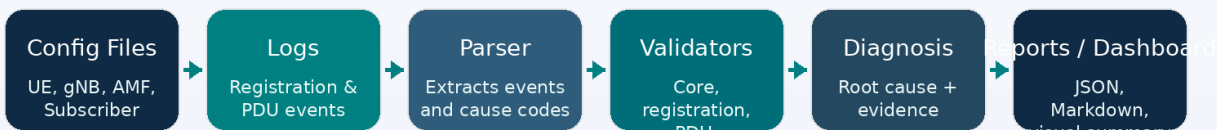
Project Objectives

- Validate whether UE, gNB, AMF, and subscriber configurations are consistent.
- Parse logs and extract useful events such as registration accept/reject and PDU session accept/reject.
- Detect common 5G slice failures including wrong DNN, wrong TAC, wrong slice, and SMF down.
- Generate readable reports and dashboard output for presentation and troubleshooting.
- Keep the system modular so more checks and log patterns can be added later.

3. System Architecture

The system is organized as a pipeline. Each module has a clear responsibility, which makes the project easier to test, explain, and extend.

5G Slice Validator - Processing Flow



Goal: reduce manual troubleshooting by combining configuration validation, log parsing, and explainable diagnosis.

Layer	Input	Responsibility
Configuration Layer	YAML files	Represents expected network setup for UE, gNB, AMF, subscriber, and runtime services.
Parsing Layer	Log files	Extracts registration, PDU session, cause codes, and assigned IP addresses.
Validation Layer	Parsed logs + configs	Checks core health, registration, PDU session, and config consistency.
Diagnosis Layer	Validation results	Correlates evidence and identifies the most probable root cause.
Reporting Layer	Final diagnosis	Generates JSON/Markdown reports and feeds the Streamlit dashboard.

4. How the Tool Works

Step	Stage	Explanation
1	Select Scenario	The user runs the project with a scenario such as success, wrong_dnn, wrong_tac, wrong_slice, or smf_down.
2	Load Inputs	The tool loads the matching log file and YAML configuration files.
3	Parse Logs	The parser extracts events such as registration accept/reject, PDU session result, cause codes, and IP addresses.
4	Run Validators	Validators check core health, registration state, PDU session status, and consistency between configs.
5	Diagnose Root Cause	The diagnosis engine combines evidence and chooses the most likely failure reason.
6	Generate Output	The final result is saved as reports and can be viewed in the dashboard.

5. Main Modules

File / Module	Role in the Project
main.py	CLI entry point that selects the scenario, runs the pipeline, prints the result, and writes reports.
scenarios.py	Maps each scenario name to its log file and configuration folder.
parser/engine.py	Reads raw logs and converts them into structured events and extracted evidence.
parser/patterns.py	Stores regular expressions and cause-code mappings used by the parser.
validator/core_health.py	Checks whether important 5G core services are running or marked as active in mock mode.
validator/registration.py	Validates whether the UE registration flow passed or failed.
validator/pdu_session.py	Checks whether the PDU session was established and whether the UE received an IP address.
validator/config_consistency.py	Compares DNN, TAC, PLMN, and S-NSSAI across UE, gNB, AMF, and subscriber configs.
validator/diagnosis.py	Combines all findings and produces root cause, confidence, evidence, and recommendations.
reports/generator.py	Generates JSON and Markdown reports for the last run.
dashboard/app.py	Streamlit interface for viewing the latest validation result visually.

6. Supported Scenarios and Results

The project includes reproducible scenarios so the demo can show both successful and failed 5G configurations.

Scenario	Purpose	Status	Detected Root Cause
success	Correct configuration and successful PDU session	PASS	No dominant failure pattern detected
wrong_dnn	UE requests a DNN not allowed by subscriber/config	FAIL	Requested DNN is not configured or not allowed
wrong_tac	Tracking Area Code mismatch	FAIL	TAC mismatch - tracking area not served by AMF
wrong_slice	Requested S-NSSAI does not match supported/subscribed slice	FAIL	Slice mismatch between UE, AMF, and subscriber
smf_down	SMF service is unavailable	FAIL	SMF is not running or not active

Demo Value

These scenarios make the project easy to present. You can run one successful case, then run failure cases to show how the tool detects the reason and gives recommendations.

7. How to Run the Project

Prerequisites

- Python 3.x installed.
- Project dependencies installed from requirements.txt.
- Terminal opened inside the project folder.

Installation and Basic Commands

Action	Command
Open project folder	cd project/5g-slice-validator
Install requirements	pip install -r requirements.txt
Run all tests	python -m pytest -q
Run success scenario	python main.py --scenario success
Run DNN failure scenario	python main.py --scenario wrong_dnn
Run all scenarios	python tools/run_all_scenarios.py
Open dashboard	streamlit run dashboard/app.py

8. Testing and Verification

The project includes automated tests to verify that the parser, configuration checks, and pipeline behave correctly. This improves reliability and makes the project easier to grade or maintain.

Test Area	Command	Result
Unit and pipeline tests	python -m pytest -q	9 passed

9. Limitations and Future Work

Current Limitations

- The current version mainly works as an offline/reproducible prototype using sample logs and YAML configs.
- The log parser is regex-based, so new log formats may require adding more patterns.
- The diagnosis engine is rule-based, which is explainable but depends on defined rules and known failure patterns.
- The dashboard is functional but still simple and can be improved visually.

Future Improvements

- Add live log monitoring for Open5GS/UERANSIM deployments.
- Allow users to upload custom logs and configuration files through the dashboard.
- Add more 5G cause-code mappings and deeper troubleshooting rules.
- Improve dashboard charts, severity indicators, and timeline view.
- Export a final PDF report directly from the tool.
- Add optional integration with real systemd services on Linux for live core health checks.

10. Conclusion

The 5G Slice Validator is a strong academic prototype for troubleshooting 5G slicing and PDU session problems. It combines configuration validation, log parsing, rule-based diagnosis, reporting, and dashboard visualization in one organized tool.

The main value of the project is that it reduces manual troubleshooting time and explains failures clearly. Instead of only showing that a scenario failed, it identifies the most likely cause, supports the answer with evidence, and gives recommendations for fixing the issue.

Final Statement

This project demonstrates how automated validation can support engineers in understanding and fixing common 5G slice deployment issues in a clear, reproducible, and professional way.

Appendix: Project Structure

5g-slice-validator/

├─ main.py

├─ scenarios.py

├─ configs/

├─ logs/

├─ parser/

├─ validator/

├─ reports/

├─ dashboard/

├─ tools/

└─ tests/